

Development Prompt — Hockey Game Analyzer

v1.0 Release-Fix Sprint

You are working on the `v1.0` release candidate of the NHL Hockey Game Analyzer.

Repository:

`https://github.com/david-turnbull/Hockey-game-analyzer/tree/v1.0`

The application is already functionally close to release. Do **not** add major new features or redesign the architecture.

This task is a focused **release-fix sprint** intended to resolve the remaining consistency, analytics-correctness, CI, and presentation issues before `v1.0` is tagged and published.

The goal is:

Make the existing v1.0 implementation internally consistent, analytically defensible, correctly validated by CI, and portfolio-ready.

Scope

Address the following five issue groups:

1. Make `LineService` use the centralized `OnIceService`
2. Remove unsafe historical-team fallback behaviour
3. Ensure the v1.0 release candidate is actually validated by CI
4. Finish README/repository presentation
5. Correct small possession and player-stat edge cases

Do not implement season dashboards, trained xG, WAR, predictions, or other new analytics during this task.

Development Principles

Follow these rules:

1. Inspect the existing implementation before changing it.
2. Make the smallest change required to solve each issue.
3. Preserve current functionality.
4. Do not redesign the application.

5. Do not introduce unnecessary dependencies.
6. Existing tests must continue to pass.
7. Every correctness fix should receive a regression test.
8. Prefer one authoritative implementation of shared analytical logic.
9. Avoid silent fallbacks that can produce plausible but incorrect hockey data.
10. Do not fabricate missing NHL information.
11. Treat this as a release stabilization sprint rather than a feature sprint.
12. Clearly document any remaining known limitations.

Before making changes, provide a concise plan listing:

- files expected to change
- why each change is required
- tests that will be added or updated
- whether any schema/database migration is required

Then implement the fixes.

Issue 1 — Make `LineService` Use `OnIceService`

Problem

The project now has a centralized:

OnIceService

which is intended to provide the authoritative answer to:

Who was on the ice at time T?

However, `LineService` still performs some of its own shift reconstruction.

This duplicates one of the most important analytical rules in the application.

Even if the current boundary handling happens to match `OnIceService`, duplicated shift logic creates a future risk that:

PossessionService

and:

LineService

could disagree about which players were actually on the ice.

1.1 Inspect Existing Line Logic

Review the current implementation of:

app/services/line_service.py

Identify all places where it:

- loads shifts
- constructs active-player timelines
- determines players active at time T
- reconstructs skater combinations
- handles shift boundaries
- filters anomalous shifts

Also inspect:

app/services/on_ice_service.py

to determine what functionality can be reused directly.

1.2 Make OnIceService Authoritative

Refactor LineService so it delegates active-player determination to OnIceService.

The final architecture should conceptually be:

```
LineService
  |
  v
OnIceService
  |
  v
Shift data
```

rather than:

```
LineService -----> Shift data
PossessionService -> OnIceService -> Shift data
```

There should be one authoritative implementation of shift presence.

1.3 Preserve Efficient Line Reconstruction

Do not introduce a severe performance regression.

If calling:

```
get_players_on_ice(...)
```

for every second creates excessive database queries, improve the design appropriately.

Acceptable options include:

- preloading valid game shifts once
- allowing `OnIceService` to operate from preloaded shifts
- creating a reusable active-player timeline
- caching active-player state for the duration of one service call

The important requirement is that the **rules remain owned by** `OnIceService`, even if performance requires a batch-oriented helper.

Do not recreate independent `[start, end)` logic inside `LineService`.

1.4 Preserve Shift Semantics

The existing convention should remain:

```
[start, end)
```

meaning:

```
start <= t < end
```

Ensure:

- outgoing player is not active at the exact end time
- incoming player is active at the exact start time
- touching shifts do not create duplicate players

1.5 Regression Tests

Add tests proving that line reconstruction uses the same active-player interpretation as `OnIceService`.

At minimum test:

```
Player A shift: [600, 640)
Player B shift: [640, 680)
```

Expected:

```
t = 639 -> Player A active
t = 640 -> Player B active
```

Also test a forward-line or defence-pair change occurring exactly at a shift boundary.

Issue 2 — Remove Unsafe Historical-Team Fallback

Problem

The project now correctly uses a historical game-roster model such as:

```
GamePlayer
```

However, `PlayerGameService` may still fall back to:

```
player.current_team_id
```

when a historical `GamePlayer` record cannot be found.

This can silently produce incorrect historical attribution after a player is traded.

Example:

```
October:  
Player -> Calgary  
  
March:  
Player traded to Vancouver  
  
Player.current_team_id = Vancouver
```

A historical October query must never infer that the player represented Vancouver in that game.

2.1 Remove Historical Dependence on Current Team

Review:

```
app/services/player_game_service.py
```

and search the entire repository for logic similar to:

```
player.current_team_id
```

used to determine the player's team in a historical game.

Historical game attribution must use:

1. `GamePlayer`, or
2. another defensible game-specific source

It must not silently use the player's present-day/current-team field.

2.2 Define Safe Fallback Behaviour

If the `GamePlayer` record is missing, use an explicit fallback hierarchy.

Recommended order:

1. `GamePlayer`
2. Valid game-specific shift team
3. Valid game-specific event/team evidence
4. Unknown / unresolved

Use only existing trustworthy data.

Do not invent a team.

If the team cannot be determined, return an explicit value such as:

Unknown

or:

None

depending on existing application conventions.

The UI/service should handle this gracefully.

2.3 Detect Data Integrity Problems

A missing `GamePlayer` relationship should ideally be treated as a data-integrity warning.

Where appropriate, update the database diagnostics so they can identify:

player appears in game events/shifts but has no `GamePlayer` row

Do not silently hide missing historical roster data.

2.4 Regression Tests

Add a traded-player test.

Example:

```
Game A:
Player 100 -> CGY

Later:
Player.current_team_id -> VAN
```

Expected:

```
Player Game Page for Game A -> CGY
```

Also add a test where the `GamePlayer` record is missing.

Expected:

- trusted game-specific fallback is used if available
- otherwise team is unresolved
- current team must not silently determine historical team

Issue 3 — Fix CI for the v1.0 Release Candidate

Problem

The repository contains GitHub Actions test automation, but the current workflow may only run on branches such as:

```
main
master
```

The active release candidate is:

```
v1.0
```

Therefore changes made directly to the release branch may not currently receive automated test validation.

The v1.0 release checklist requires CI to pass.

3.1 Review GitHub Actions Configuration

Inspect:

```
.github/workflows/tests.yml
```

or the existing equivalent.

Review:

- push branch filters
- pull-request branch filters
- Python version
- dependency installation
- pytest command
- required environment configuration

3.2 Ensure Release Branch Testing

Configure CI so the `v1.0` release candidate is automatically tested.

At minimum CI should run for:

```
push -> v1.0  
pull request -> main
```

and ideally continue supporting normal development branches appropriately.

Do not overcomplicate the workflow.

3.3 Recommended Release Flow

Prepare the project for the following release workflow:

```
v1.0 branch  
|  
v  
Automated CI  
|
```

```
    v
Pull Request to main
    |
    v
Automated CI
    |
    v
Merge
    |
    v
Tag tested commit as v1.0
    |
    v
Create GitHub Release
```

Do not automatically create the tag or GitHub Release unless explicitly instructed.

3.4 Clean Installation Test

Ensure GitHub Actions creates a clean environment rather than relying on developer-local state.

CI should:

```
checkout repository
install Python
install dependencies
run tests
```

If the test suite requires temporary database initialization, ensure this happens automatically.

Do not depend on a developer's existing SQLite database.

3.5 CI Documentation

Add a test/CI section to the README if not already clear.

Include:

```
pytest
```

and explain that GitHub Actions runs the test suite automatically.

If practical, add the GitHub Actions status badge to the README.

Issue 4 — Finish README and Repository Presentation

Problem

The README has improved substantially but still contains presentation gaps.

The repository should visually communicate the value of the project to someone who may spend only 1–2 minutes initially reviewing it.

This is particularly important for portfolio use.

4.1 Replace Screenshot Placeholders

Inspect the README for references such as:

```
docs/screenshots/game-overview.png
```

If actual screenshots exist:

- move or reference them correctly
- verify they render on GitHub
- use meaningful captions

If screenshots do not exist, do not fabricate them.

Instead:

1. create the required screenshot directory
2. document the exact screenshots still required
3. ensure README references are not broken

Preferred screenshots:

```
Game selector  
Game overview
```

```
Shot map
Player Game Page
Lines / possession analysis
```

If automated screenshot generation is practical with the existing application, it may be used, but do not introduce a complex browser-testing dependency solely for this purpose.

4.2 Verify Mermaid Rendering

Review the architecture diagram.

Make sure it uses valid GitHub Mermaid syntax such as:

```
```mermaid

flowchart TD
 A[NHL API] --> B[Raw JSON Cache]
 B --> C[Transform + Validate]
 C --> D[SQLite / SQLAlchemy]
 D --> E[Service Layer]
 E --> F[Flask Routes / API]
 F --> G[Analytics UI]
```

Do not leave the architecture diagram displayed as plain source code if GitHub can render it.

---

### # 4.3 Improve README Opening

The first screen of the README should quickly communicate:

#### ### What it is

An NHL post-game analysis platform.

#### ### Who it is for

Hockey fans and analysts interested in understanding more than the traditional box score.

#### ### What it does

Examples:

```
```text  
Shot analysis  
Player game analysis  
Shift reconstruction  
5v5 possession  
Forward lines  
Defence pairings  
Game-event timelines  
Experimental xG
```

Keep language precise and professional.

4.4 Include a Clear v1.0 Status

Until the release is actually tagged, describe the branch as:

```
v1.0 Release Candidate
```

Do not describe it as a published release if no release/tag exists.

After the release is created, the README can be updated separately.

4.5 Ensure Limitations Remain Visible

Retain or strengthen the limitations section.

At minimum mention:

- NHL public-data limitations
- shift timestamps are not frame-level tracking
- reconstructed on-ice state depends on source shift data
- experimental xG uses hand-selected coefficients
- full-season/multi-season validation is future work

Do not hide methodological limitations.

Issue 5 — Correct Possession Empty-Sample Behaviour

Problem

Current possession calculations may return:

50.0%

when:

$CF + CA = 0$

or:

$FF + FA = 0$

This implies evenly split possession when there were actually no observations.

The mathematically correct state is undefined / unavailable.

5.1 Change Zero-Denominator Behaviour

For:

$CF\% = CF / (CF + CA)$

if:

$CF + CA == 0$

return:

None

or another explicit missing/undefined representation already used by the application.

Do the same for Fenwick.

Do not return `50.0`.

5.2 Update UI Formatting

Ensure the UI renders undefined values cleanly.

Examples:

```
N/A
-
No 5v5 events
```

Choose one consistent convention.

Do not display:

```
None%
```

or:

```
0%
```

unless zero is actually the correct calculated result.

5.3 Regression Tests

Add tests for:

```
CF = 0
CA = 0
=> CF% = undefined
```

and:

```
FF = 0  
FA = 0  
=> FF% = undefined
```

Also verify:

```
CF = 0  
CA = 5  
=> CF% = 0%
```

because this is a real zero rather than missing data.

Issue 6 — Fix Average Shift Calculation

Problem

The Player Game Page appears to exclude anomalous shifts from total TOI but may divide by the total number of shifts, including invalid/anomalous ones, when calculating average shift length.

Example:

```
10 recorded shifts  
2 invalid shifts  
8 valid shifts used for TOI
```

Average shift should use:

```
valid TOI / valid shift count
```

not:

```
valid TOI / 10
```

6.1 Use a Valid Shift Set

Create one clearly defined valid shift collection.

Use the same collection for:

```
TOI
valid shift count
average shift duration
```

If the page also displays total raw shift records, distinguish that metric explicitly.

6.2 Avoid Duplicate Validation Rules

If shift-validity rules already exist in:

```
OnIceService
```

or another centralized utility, reuse them.

Do not create another independent definition of:

```
valid shift
```

inside `PlayerGameService`.

6.3 Regression Tests

Test something similar to:

```
Shift 1: 30 seconds
Shift 2: 40 seconds
Shift 3: invalid zero-duration
```

Expected:

```
TOI = 70 seconds
Valid shifts = 2
Average shift = 35 seconds
```

The invalid shift must not reduce the calculated average.

Issue 7 — Re-run Database Integrity Checks

After the fixes, run the database diagnostics against the existing dataset.

Report:

```
games
teams
players
GamePlayer records
events
shots
shifts
```

and specifically check:

```
orphan records
missing GamePlayer relationships
historical team mismatches
duplicate GamePlayer rows
negative shifts
zero-duration shifts
invalid period states
invalid manpower states
missing shooter IDs
```

Do not suppress warnings merely to achieve PASS status.

Issue 8 — Run the Full Test Suite

Run all tests.

Report:

```
tests passed
tests failed
tests skipped
```

Do not state that all tests pass unless they were actually executed.

If tests fail, investigate and fix issues within the scope of this sprint.

Do not weaken meaningful tests just to obtain a passing result.

Issue 9 — Update Release Checklist

Review:

`release_checklist.md`

Update it so each item accurately reflects the final implementation.

Verify at minimum:

- ☐ historical roster attribution is authoritative
- ☐ no silent historical fallback to `current_team_id`
- ☐ true 5v5 filtering works
- ☐ half-open shift semantics are consistent
- ☐ `OnIceService` is authoritative
- ☐ `LineService` delegates on-ice logic appropriately
- ☐ player TOI/average-shift calculations use valid shifts
- ☐ undefined possession percentages display as N/A
- ☐ tests pass
- ☐ GitHub Actions runs on the release candidate
- ☐ README renders correctly
- ☐ screenshots are real or clearly identified as pending
- ☐ database diagnostics run successfully

Do not mark criteria complete unless verified.

Do Not Implement

Do not implement during this task:

Season dashboard
Multi-season database
Trained xG
XGBoost
WAR
RAPM
Game predictions

```
Player projections
Betting analytics
Video integration
Live-game support
User authentication
Major frontend redesign
Cloud deployment
New visualization suite
```

These belong to later versions.

Required Tests

At minimum ensure the suite includes coverage for:

```
LineService uses centralized on-ice semantics
Line change exactly at shift boundary
Historical traded-player attribution
Missing GamePlayer does not silently use current_team_id
Zero Corsi denominator returns undefined
Zero Fenwick denominator returns undefined
True 0% Corsi remains 0%
Average shift excludes anomalous shifts
Valid shift count matches TOI calculation
Existing Player Game Page behaviour
Existing possession calculations
Existing line combinations
Existing defensive pairings
Existing NHL fixture regression tests
```

Definition of Done

The release-fix sprint is complete when:

- ☐ `LineService` no longer owns independent on-ice rules
- ☐ `OnIceService` remains the authoritative shift-presence implementation
- ☐ historical player-team attribution never silently uses current team
- ☐ missing historical roster relationships are handled explicitly
- ☐ CI runs for the v1.0 release candidate
- ☐ CI can run from a clean environment
- ☐ README architecture diagram renders properly

- [] screenshot placeholders are resolved or explicitly documented
- [] repository describes v1.0 accurately as a release candidate until tagged
- [] possession percentages return N/A when there are no observations
- [] actual 0% possession remains distinguishable from N/A
- [] average shift uses only valid shifts
- [] database diagnostics run successfully
- [] all automated tests pass
- [] release checklist accurately reflects the implementation
- [] no major new features were introduced

Final Deliverable

When finished, provide a concise release-readiness report.

1. Files Changed

List the changed files and purpose of each.

2. On-Ice Refactor

Explain:

- what logic moved or was removed from `LineService`
- how `OnIceService` is now reused
- whether any performance optimization was required

3. Historical Team Attribution

Explain the final lookup order.

State explicitly whether any code still uses:

```
player.current_team_id
```

for historical game attribution.

The expected answer should be:

```
No
```

unless there is a clearly documented non-historical use.

4. Analytics Corrections

Report the final behaviour for:

```
CF + CA = 0
FF + FA = 0
average shift duration
invalid shifts
```

5. CI

Report:

- workflow triggers
- Python version
- pytest command
- whether the workflow has passed

6. Database Integrity

Report actual diagnostic results.

7. Test Results

Report:

```
Passed:
Failed:
Skipped:
```

using actual results.

8. README / Portfolio State

Confirm:

- architecture diagram renders
- screenshots status
- release-candidate wording
- limitations are documented

9. Remaining Issues

List only genuine remaining v1.0 blockers or warnings.

10. Release Recommendation

Return exactly one of:

```
READY TO TAG v1.0  
READY TO TAG v1.0 WITH MINOR WARNINGS  
NOT READY TO TAG v1.0
```

Then explain why.

Do not create the Git tag or GitHub Release unless explicitly instructed.

Expected Next Step

If the release recommendation is:

```
READY TO TAG v1.0
```

the next project phase should **not** be another v1 feature.

The next milestone should be:

v2 — Full-Season / Season Analytics

Its initial focus should be:

```
Full-season NHL ingestion  
Season-wide data validation  
Team season dashboard  
Player game-to-game trends  
Line/pairing trends  
Performance benchmarking  
Preparation for a trained xG dataset
```

Do not begin v2 during this task.